

FULLSTACK D3

and DATA VISUALIZATION



FULLSTACK.io

AMELIA WATTENBERGER

Fullstack Data Visualization with D3

Build Beautiful Data Visualizations and Dashboards with D3

Written by Amelia Wattenberger

Edited by Nate Murray

© 2019 Fullstack.io

All rights reserved. No portion of the book manuscript may be reproduced, stored in a retrieval system, or transmitted in any form or by any means beyond the number of purchased copies, except for a single backup or archival copy. The code may be used freely in your projects, commercial or otherwise.

The authors and publisher have taken care in preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

Published by Fullstack.io.

Contents

Book Revision	1
Bug Reports	1
Be notified of updates via Twitter	1
We'd love to hear from you!	1
Making Your First Chart	1
Getting started	1
Loading the weather dataset	3
Getting a server running	5
Looking at our data	6
Setting up our line chart	8
Drawing our chart	10
Creating our workspace	12
Adding an SVG element	13
Creating our bounding box	15
Creating our scales	17
Drawing the line	23
Drawing the axes	27
Making a Scatterplot	33
Intro	33
Deciding the chart type	33
Steps in drawing any chart	34
Access data	37
Create chart dimensions	37
Draw canvas	41
Create scales	43
The concept behind scales	43

CONTENTS

Finding the extents	44
Draw data	47
Data joins	50
Data join exercise	55
.join()	58
Draw peripherals	59
Initialize interactions	63
Looking at our chart	64
Extra credit: adding a color scale	64
Making a Bar Chart	68
Deciding the chart type	68
Histogram	69
Chart checklist	72
Access data	73
Create dimensions	73
Draw canvas	75
Create scales	76
Creating Bins	77
Creating the y scale	80
Draw data	81
Adding Labels	85
Extra credit	88
Draw peripherals	92
Set up interactions	93
Looking at our chart	93
Extra credit	94
Accessibility	99
Animations and Transitions	104
SVG <animate>	104
CSS transitions	106
Using CSS transition with a chart	111
d3.transition	116
Lines	128
Interactions	135

CONTENTS

d3 events	135
Don't use fat arrow functions	139
Destroying d3 event listeners	140
Bar chart	141
Scatter plot	156
Voronoi	161
Changing the hovered dot's color	168
Line chart	171
d3.scan()	174
Making a map	181
Digging in	182
What is GeoJSON?	182
Access data	184
Our dataset	187
Create chart dimensions	190
What is a projection?	190
Which projection should I use?	191
Finishing creating our chart dimensions	194
Draw canvas	197
Create scales	198
Draw data	201
Draw peripherals	207
Drawing a legend	207
Marking my location	217
Set up interactions	220
Wrapping up	226
Data Visualization Basics	227
Types of data	229
Qualitative Data	231
Quantitative Data	233
Ways to visualize a metric	234
Size	235
Position	236
Color	237

CONTENTS

Putting it together	238
Chart design	238
Simplify, simplify, simplify	238
Annotate in-place	239
Add enhancements, but not too many	240
Example redesign	240
Colors	246
Color scales	247
1. Representing a category	247
Custom color scales	250
Creating our own colors	254
keywords	254
rgb	255
hsl	256
hcl	257
d3-color	259
Color tips	260
Wrapping up	264
Common charts	266
Chart types	267
Timeline	268
Heatmap	271
Radar	273
Scatter	275
Pie charts & Donut charts	282
Histogram	284
Box plot	288
Conclusion	291
Dashboard Design	293
What is a dashboard?	293
Showing a simple metric	294
Dealing with dynamic data	301
Data states	301
Dancing numbers	305

CONTENTS

Designing tables	308
Designing a dashboard layout	316
Have a main focus	316
Keep the number of visible metrics small	317
Let the user dig in	318
Deciding questions	319
How familiar are users with the data?	319
How much time do users have to spend?	320
Complex Data Visualizations	321
Marginal Histogram	322
Chart bounds background	325
Equal domains	327
Color those dots	329
Mini histograms	333
Static polish	341
Adding a tooltip	344
Histogram hover effects	347
Adding a legend	353
Highlight dots when we hover the legend	361
Mini hover histograms	373
Radar Weather Chart	380
Getting set up	381
Accessing the data	382
Creating our scales	384
Adding gridlines	384
Draw month grid lines	385
Draw month labels	395
Adding temperature grid lines	399
Adding freezing	406
Adding the temperature area	408
Adding the UV index marks	412
Adding the cloud cover bubbles	414
Adding the precipitation bubbles	421
Adding annotations	425

CONTENTS

Adding the tooltip	433
Wrapping up	446
Animated Sankey Diagram	448
Getting set up	449
Accessing our data	449
Accessing sex variables	451
Accessing education variables	452
Accessing socioeconomic status variables	453
Stacking probabilities	453
Generating a person	457
Drawing the paths	460
X scale	461
Y scales	462
Drawing the paths	463
Labeling the paths	469
Start labels	469
End labels	472
Drawing the ending markers	473
Drawing people	478
Positioning our people	482
Updating our peoples' y-positions	484
Adding jitter	487
Hiding people off-screen	488
Adding color	490
Creating a color scale	490
Adding a color key	491
Coloring our markers	492
Adding a filter to our markers	493
Giving our people ids	495
Showing ending numbers	497
Updating the ending values	500
Label our ending bars	503
Additional steps	507
Wrapping up	507

CONTENTS

Using D3 With React	508
React.js	509
Digging in	510
Access data	512
Create dimensions	513
Draw canvas	515
Create scales	518
Draw data	519
Draw peripherals	522
Axes, take two	526
Set up interactions	533
Finishing up	534
Using D3 With Angular	536
Angular	537
Digging in	538
Access data	540
Create dimensions	542
Updating dimensions on window resize	546
Draw canvas	547
Using our chart component	550
Create scales	551
When to update our scales?	552
Draw data	553
Using our line component	557
Draw peripherals	558
Axes, take two	561
Re-creating our axis component	562
Set up interactions	568
Finishing up	568
D3.js	571
What did we cover?	572
What did we not cover?	574
Going forward	577
How was your experience?	578

CONTENTS

Appendix	579
A. Generating our own weather data	579
Chrome's Color Contrast Tool	580
B. Chart-drawing checklist	583
C. SVG elements cheat sheet	585
Changelog	586
Revision 15	586
04-01-2020	586
Revision 14	588
03-11-2020	588
Revision 13	588
01-08-2020	588
Revision 12	588
12-09-2019	588
Revision 11	589
10-11-2019	589
Revision 10	589
09-29-2019	589
Revision 9	590
08-28-2019	590
Revision 8	590
07-04-2019	590
Revision 7	591
06-14-2019	591
Revision 6	592
06-06-2019	592
Revision 5	592
06-03-2019	592
Revision 4	593
05-24-2019	593
Revision 3	593
05-17-2019	593

Book Revision

- Revision 1 - 2019-05-12
- Revision 2 - 2019-05-14
- Revision 3 - 2019-05-17
- Revision 4 - 2019-05-30
- Revision 5 - 2019-06-03
- Revision 6 - 2019-06-06
- Revision 7 - 2019-06-14
- Revision 8 - 2019-07-04
- Revision 9 - 2019-08-28
- Revision 10 - 2019-09-29
- Revision 11 - 2019-10-11
- Revision 12 - 2019-12-09
- Revision 13 - 2020-01-08
- Revision 14 - 2020-03-11
- Revision 15 - 2020-04-01

Bug Reports

If you'd like to report any bugs, typos, or suggestions just email us at: us@fullstack.io.

Be notified of updates via Twitter

If you'd like to be notified of updates to the book on Twitter, follow us at [@fullstackio](https://twitter.com/fullstackio)¹.

We'd love to hear from you!

Did you like the book? Did you find it helpful? We'd love to add your face to our list of testimonials on the website! Email us at: us@fullstack.io².

¹<https://twitter.com/fullstackio>

²<mailto:us@fullstack.io>

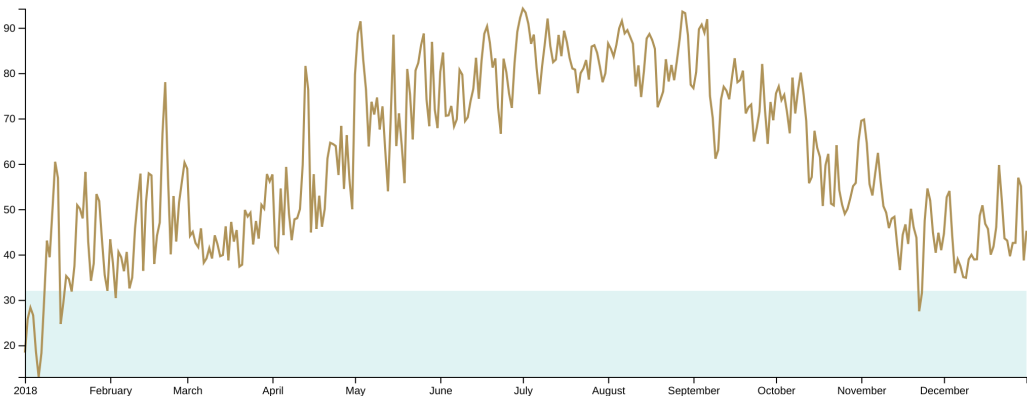
Making Your First Chart

Many books begin by talking about theory and abstract concepts. This is not one of those books. We'll dig in and create real charts right away! Once you're comfortable making your own charts, we'll discuss how to integrate into websites, data visualization fundamentals, and practical tips for chart design, along with other goodies.

Getting started

To start, let's make a line chart. Line charts are a great starting place because of their popularity, but also because of their simplicity.

In this chapter, we'll create a line chart that plots the daily temperature. Here's what our line chart will look like when we're finished:



Finished line graph

In the tutorial below, don't worry too much about the details! We're just going to get our hands dirty and write the code to build this line chart. This will give us a good foundation to dive deeper into each concept in Chapters 2 and 3, in which we'll create more complex charts.

The dataset we'll be analyzing contains 365 days of daily weather metrics. To make it easy, we've provided a JSON file with this data in the code download folder named `nyc_weather_data.json`. This file includes 2018 data for New York City.

We recommend that you create a dataset for your own location to keep the data tangible, plus you'll discover new insights about where you live! Refer to **Appendix A** for instructions — it should only take a few minutes and the charts we make together will be uniquely yours.

If you're using the provided dataset, rename the file to `my_weather_data.json`. This way, our code examples will know where to find the weather data.

Let's get our webpage up and running. Find the `index.html` file and open it in your browser. The url will start with `file:///`. This is a very simple webpage — we're rendering one element and loading two javascript files.

`code/01-making-your-first-chart/completed/index.html`

```
7      <div id="wrapper"></div>
8
9      <script src="../../../d3.v5.js"></script>
10     <script src="./chart.js"></script>
```

The page should be blank except for one `div` with an `id` of `wrapper` — this is where our chart will live.

The first script that we're loading is `d3.js` — this will give us access to the entire library.

At this point, we want access to the whole library, but `d3.js` is made up of at least thirty modules. Later, we'll discuss how to import only the necessary modules to keep your build lean.

Next, our `index.html` file loads the javascript file in which we'll write our chart code: `chart.js`.

Let's open up the `01-making-your-first-chart/draft/chart.js` file in a code editor and dig in.

If you don't already have a code editor, any program that lets you open a file and edit the text will do! I personally use **Visual Studio Code**^a, which I recommend — it's straightforward enough for beginners, but has many configuration options and extensions for power users.

^a<https://code.visualstudio.com/>

We don't have much text in here to start with.

```
async function drawLineChart() {  
    // write your code here  
}
```

```
drawLineChart()
```

The only thing happening so far is that we're defining a function named `drawLineChart()` and running it.

Loading the weather dataset

The first step to visualizing any dataset is understanding its structure. To get a good look at our data, we need to import it into our webpage. To do this, we will load the JSON file that holds our data.

D3.js has methods for fetching and parsing files of different formats in the **d3-fetch**³ module, for example, `d3.csv()`, `d3.json()`, and `d3.xml()`. Since we're working with a JSON file, we want to pass our file path to `d3.json()`.

Let's create a new variable named `dataset` and load it up with the contents of our JSON file.

³<https://github.com/d3/d3-fetch>

code/01-making-your-first-chart/completed/chart.js

```
4  const dataset = await d3.json("../..../my_weather_data.json")
```

`await` is a JavaScript keyword that will **pause the execution of a function until a Promise is resolved**. This will only work within an `async` function — note that the `drawLineChart()` function declaration is preceded by the keyword `async`.

Don't be overwhelmed by the words **Promise**, **async**, or **await** — this just means that any code (within the function) after `await d3.json("../..../my_weather_data.json")` will wait to run until `dataset` is defined. If you're curious and want to learn more, here is a great resource on [Promises in JavaScript](https://www.youtube.com/watch?v=QO4NXhWo_NM)^a.

^ahttps://www.youtube.com/watch?v=QO4NXhWo_NM



If you see a `SyntaxError: Unexpected end of JSON input` error message, check your `my_weather_data.json` file. It might be empty or corrupted. If so, re-generate your custom data or copy the `nyc_weather_data.json` file.

Now when we load our webpage we should get a CORS error in the console.

```
✖ ▶ Fetch API cannot load file:///C:/Users/watte/Development/repos/fullstack-d3-book/manuscript/code/src/1-making-your-first-charts/my_weather_data.json. URL scheme must be "http" or "https" for CORS request. d3.js:5908
✖ ▶ Uncaught (in promise) TypeError: Failed to fetch d3.js:5908
    at Object.json (d3.js:5908)
    at drawLineWeather (drawLineWeather.js:10)
    at drawLineWeather.js:127
```

CORS error

CORS is short for Cross-Origin Resource Sharing, a mechanism used to restrict requests to another domain. **We'll need to start a server to get around this safety restriction.**

Getting a server running

Thankfully, there are simple ways to spin up a simple static server. We'll walk through two different ways (**node.js** or **python**) — choose whichever one you're more comfortable with. **Note that only one of these options is necessary.**

a. node.js

*I would recommend using this method because it has **live reload** built in, meaning that our page will update when we save our changes. No page refresh necessary!*

If you don't have **node.js** installed, take a minute to install it ([instructions here](#)⁴). You can check whether or not **node.js** is already installed by using the `node -v` command in your terminal — if it responds with a version number, you're good to go! **node.js** should also come with **npm**, which is short for *Node Package Manager*

Once **node.js** and **npm** are installed, run the following command in your terminal.

```
npm install -g live-server
```

This will install **live-server**⁵, a simple static server that has live reload built-in. To start your server, run `live-server` in your terminal from the root `/code` folder — it will even open a new browser window for you!

a. python

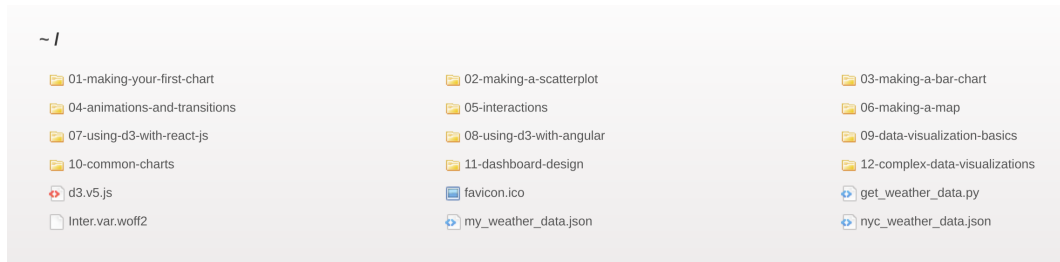
If you have **python** (version 3) installed already, you can use the Python 3 http server instead. Start it up by running the command `python -m http.server 8080` in your terminal from the root `/code` folder.

The particular server doesn't matter — the key idea is that if you want to load a file from JavaScript, you need to do it from a webserver, and these tools are an easy solution for a development environment. Make sure that you are in the root `/code` folder when you start either server.

⁴<https://docs.npmjs.com/downloading-and-installing-node-js-and-npm>

⁵<https://github.com/tapio/live-server>

Now we should have a server on port 8080. Load localhost:8080⁶ in your web browser and you'll see a directory of code for each chapter, which looks something like this:



Directory screenshot

Click on **01-making-your-first-chart** or go to <http://localhost:8080/01-making-your-first-chart/draft/>⁷ to load this chapter's `index.html` file.

For all of our code examples, there will be a finished version in a sibling `/completed` folder — in this chapter, look at `/code/01-making-your-first-chart/completed/` if you want a reference. Or view to completed chart at <http://localhost:8080/01-making-your-first-chart/completed/>.

We'll still see a blank page since we haven't drawn anything on our page yet, but that error should be gone!

Looking at our data

Going back to our code, let's log our dataset to the console. We can do that by adding the following line of code right after we create our `dataset` file.

```
console.log(dataset)
```

We can see that our dataset is array of objects, with one object per day.

⁶<http://localhost:8080>

⁷<http://localhost:8080/01-making-your-first-chart/draft/>

Since each day seems to have the same structure, let's delete the last line and instead log a single data point to get a clearer view.

We can use `console.table()` here, which is a great function for looking at array or object values — as long as there aren't too many!

```
console.table(dataset[0])
```

chart.js:6

(index)	Value
time	1514782800
summary	"Clear throughout the day."
icon	"clear-day"
sunriseTime	1514809280
sunsetTime	1514842810
moonPhase	0.48
precipIntensity	0
precipIntensityMax	0
precipProbability	0
temperatureHigh	18.39
temperatureHighTime	1514836800
temperatureLow	12.23
temperatureLowTime	1514894400
apparentTemperatureHigh	17.29
apparentTemperatureHighTime	1514844000
apparentTemperatureLow	4.51
apparentTemperatureLowTime	1514887200
dewPoint	-1.67
humidity	0.54
pressure	1028.26
windSpeed	4.16
windGust	13.98
windGustTime	1514829600
windBearing	309
cloudCover	0.02
uvIndex	2
uvIndexTime	1514822400
visibility	10
temperatureMin	6.17
temperatureMinTime	1514808000
temperatureMax	18.39
temperatureMaxTime	1514836800
apparentTemperatureMin	-2.19
apparentTemperatureMinTime	1514808000
apparentTemperatureMax	17.29
apparentTemperatureMaxTime	1514844000
date	"2018-01-01"

» Object

Our dataset

We have lots of information for each day! We can see metadata (date, time, summary) and details about that day's weather (cloudCover, sunriseTime, temperatureMax, etc). If you want to read more about each metric, check out [The Dark Sky API docs](https://darksky.net/dev/docs#data-point)⁸.

⁸<https://darksky.net/dev/docs#data-point>

Setting up our line chart

Let's dig in by looking at `temperatureMax` over time. Our timeline will have two axes:

- a y axis (vertical) on the left comprised of max temperature values
- an x axis (horizontal) on the bottom comprised of dates

To grab the correct metrics from each data point, we'll need *accessor* functions. **Accessor functions convert a single data point into the metric value.**

Lets try it out by creating a `yAccessor` function that will take a data point and return the max temperature.

If you think of a **dataset** as a table, a **data point** would be a row in that table. In this case, a **data point** represents an item in our dataset array: an object that holds the weather data for one day.

We will use `yAccessor` for plotting points on the y axis.

Looking at the data point in our console, we can see that a day's max temperature is located on the object's `temperatureMax` key. To access this value, our `yAccessor` function looks like this:

`code/01-making-your-first-chart/completed/chart.js`

```
6  const yAccessor = d => d.temperatureMax
```

Next, we'll need an `xAccessor` function that will return a point's date, which we will use for plotting points on the x axis.

```
const xAccessor = d => d.date
```

But look closer at the data point `date` value - notice that it is a *string* (eg. "2018-12-25"). Unfortunately, this string won't make sense on our x axis. How could we know how far "2018-12-25" is from "2018-12-29"?

We need to convert the string into a JavaScript Date, which is an object that represents a single point in time. Thankfully, d3 has a [d3-time-format](https://github.com/d3/d3-time-format)⁹ module with methods for parsing and formatting dates.

The `d3.timeParse()` method...

- takes a string specifying a date format, and
- outputs a function that will parse dates of that format.

For example, `d3.timeParse("%Y")` will parse a string with just a year (eg. "2018").

Let's create a date parser function and use it to transform our date strings into date objects:

code/01-making-your-first-chart/completed/chart.js

```
7  const dateParser = d3.timeParse("%Y-%m-%d")
8  const xAccessor = d => dateParser(d.date)
```

Great! Now when we call `xAccessor(dataset[0])`, we'll get the first day's date.

If you look up d3 examples, you won't necessarily see accessor functions used. When I first started learning d3, I never thought about using them. Since then, I've learned my lesson and paid the price of painstakingly picking through old code and updating individual lines. I want to save you that time so you can spend it making even more wonderful charts.

Defining accessor functions might seem like unnecessary overhead right now, especially with this simple example. However, creating a separate function to read the values from our data points helps us in a few ways.

⁹<https://github.com/d3/d3-time-format>

- **Easy changes:** every chart is likely to change at least once — whether that change is due to business requirements, design, or data structure. These changing requirements are especially prevalent when creating dashboards with dynamic data, where you might need to handle a new edge case two months later. Having the accessor functions in one place at the top of a chart file makes them easy to update throughout the chart.
- **Documentation:** having these functions at the top of a file can give you a quick reminder of what metrics the chart is plotting and the structure of the data.
- **Framing:** sitting down with the data and planning what metrics we'll need to access is a great way to start making a chart. It's tempting to rush in, then two hours later realize that another type of chart would be better suited to the data.

Now that we know how to access our dataset, we need to **prepare to draw our chart**.

Drawing our chart

When drawing a chart, there are two containers whose dimensions we need to define: the wrapper and the bounds.

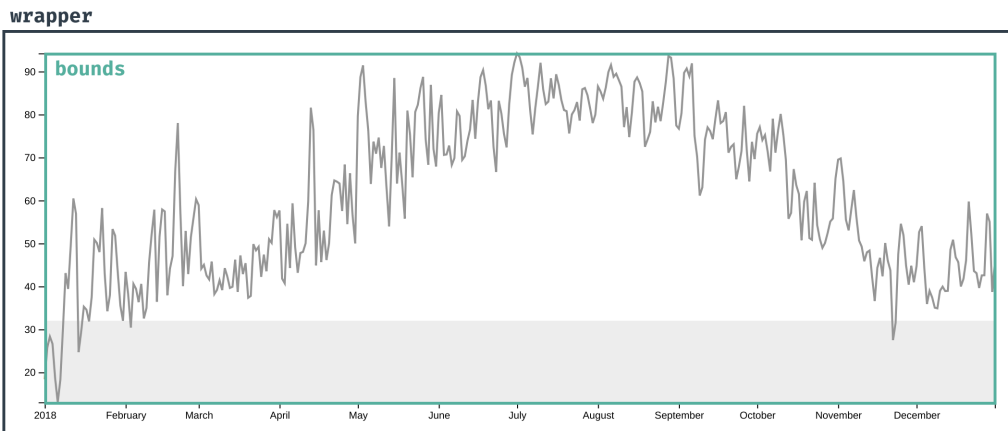


Chart dimensions

The **wrapper** contains the entire chart: the data elements, the axes, the labels, etc. Every SVG element will be contained inside here.

The **bounds** contain all of our data elements: in this case, our line.

This distinction will help us separate the amount of space we need for extraneous elements (axes, labels), and let us focus on our main task: plotting our data. One reason this is so important to define up front is the inconsistent and unfamiliar way SVG elements are sized.

When adding a chart to a webpage, we start with the amount of space we have available for the chart. Then we decide how much space we need for the margins, which will accommodate the chart axes and labels. What's left is how much space we have for our data elements.

We will rarely have the option to decide how large our timeline is and then build up from there. Our charts will need to be accommodating of window sizes, surrounding text, and more.

While **wrapper** and **bounds** isn't terminology that you'll see in widespread use, it will be helpful for reference in this book. Defining these concepts also helps with thinking about chart dimensions and remembering to make space for your axes.

Let's define a dimensions object that will contain the size of the wrapper and the margins. We'll have one margin defined for each side of the chart: top, right, bottom, and left. For consistency, we'll mimic the order used for CSS properties.

code/01-making-your-first-chart/completed/chart.js

```
12  let dimensions = {
13    width: window.innerWidth * 0.9,
14    height: 400,
15    margin: {
16      top: 15,
17      right: 15,
18      bottom: 40,
19      left: 60,
20    },
21  }
```

We want a small `top` and `right` margin to give the chart some space. The line or the y axis might overflow the chart bounds. We'll want a larger `bottom` and `left` margin to create room for our axes.

Let's compute the size of our **bounds** and add that to our `dimensions` object.

code/01-making-your-first-chart/completed/chart.js

```
22  dimensions.boundedWidth = dimensions.width
23    - dimensions.margin.left
24    - dimensions.margin.right
25  dimensions.boundedHeight = dimensions.height
26    - dimensions.margin.top
27    - dimensions.margin.bottom
```

Creating our workspace

Now we're set up and ready to start updating our webpage!

To add elements to our page, we'll need to specify an existing element that we want to append to.

Remember the `#wrapper` element already populated in `index.html`? One of d3's modules, [d3-selection](https://github.com/d3/d3-selection)¹⁰, has helper functions to select from and modify the DOM.

We can use `d3.select()`, which accepts a CSS-selector-like string and returns the first matching DOM element (if any). If you're unfamiliar with CSS selector syntax, there are three main types of selectors:

- you can select all elements with a class name (`.class`)
- you can select all elements with an id (`#id`), or
- you can select all elements of a specific node type (`type`).

¹⁰<https://github.com/d3/d3-selection>

If you've ever used jQuery or written CSS selectors, these selector strings will be familiar.

```
const wrapper = d3.select("#wrapper")
```

Let's log our new wrapper variable to the console to see what it looks like.

```
console.log(wrapper)
```

We can see that it's a d3 selection object, with `_groups` and `_parents` keys.

```
▼ Selection {_groups: Array(1), _parents: Array(1)} ⓘ  
  ▼ _groups: Array(1)  
    ▶ 0: [div#wrapper]  
      length: 1  
    ▶ __proto__: Array(0)  
  ▶ _parents: [html]  
  ▶ __proto__: Object
```

chart selection

d3 selection objects are a subclass of `Array`. They have a lot of great methods that we'll explore in depth later - what's important to us right now is the `_groups` list that contains our `#wrapper` div.

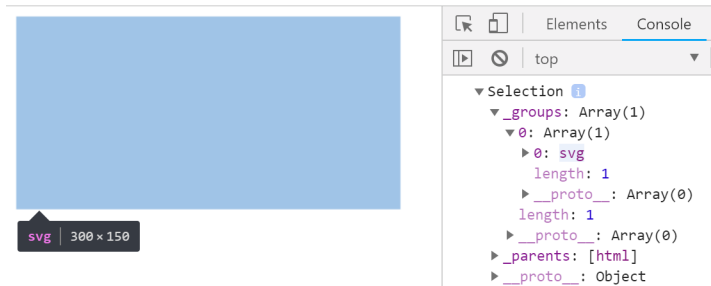
Adding an SVG element

Our wrapper object also has methods for manipulating the linked DOM element — let's use its `append` method to add a new SVG element.

```
const wrapper = d3.select("#wrapper")  
const svg = wrapper.append("svg")
```

If we log `svg` to the console, we'll see that it looks like our wrapper object. However, if we expand the `_groups` key, we'll see that the linked element is our new `<svg>` element.

One trick to make sure we're grabbing the correct element is to hover the logged DOM element. If we expand the `_groups` object and hover over the `<svg>` element, the browser will highlight the corresponding DOM element on the webpage.



svg selection with hover

On hover, the browser will also show the element's size: 300px by 150px. This is the default size for SVG elements in Google Chrome, but it will vary between browsers and even browser versions. SVG elements don't scale the way most DOM elements do — there are many rules that will be unfamiliar to an experienced web developer.

To maintain control, let's tell our `<svg>` element what size we want it to be.

d3 selection objects have an `.attr()` method that will add or replace an attribute on the selected DOM element. The first argument is the attribute name and the second argument is the value.



The value argument to `.attr()` can either be a constant, which is all we need right now, or a function, which we'll cover later.

```
const wrapper = d3.select("#wrapper")
const svg = wrapper.append("svg")
svg.attr("width", dimensions.width)
svg.attr("height", dimensions.height)
```

Most d3-selection methods will return a selection object.

- any method that selects or creates a new object will return the new selection

- any method that manipulates the current selection will return the same selection

This allows us to keep our code concise by chaining when we're using multiple methods. For example, we can rewrite the above code as:

```
const wrapper = d3.select("#wrapper")
const svg = wrapper.append("svg")
    .attr("width", dimensions.width)
    .attr("height", dimensions.height)
```

In this book, we'll follow the common d3 convention of using 4 space indents for methods that return the same selection. This will make it easy to spot when our selection changes.

Since we're not going to re-use the `svg` variable, we can rewrite the above code as:

code/01-making-your-first-chart/completed/chart.js

```
31  const wrapper = d3.select("#wrapper")
32    .append("svg")
33    .attr("width", dimensions.width)
34    .attr("height", dimensions.height)
```

When we refresh our `index.html` page, we should now see that our `<svg>` element is the correct size. Great!

Creating our bounding box

Our SVG element is the size we wanted, but we want our chart to respect the margins we specified.

Let's create a **group** that shifts its contents to respect the top and left margins so we can deal with those in one place.

Any elements inside of an `<svg>` have to be SVG elements (with the exception of `<foreignObject>` which is fiddly to work with). Since we'll be inserting new chart elements inside of our `<svg>`, we'll need to use SVG elements for the rest of the chart.

The `<g>` SVG element is not visible on its own, but is used to group other elements. Think of it as the `<div>` of SVG — a wrapper for other elements. We can draw our chart inside of a `<g>` element and shift it all at once using the CSS transform property.

d3 selection objects have a `.style()` method for adding and modifying CSS styles. The `.style()` method is invoked similarly to `.attr()` and takes a key-value pair as its first and second arguments. Let's use `.style()` to shift our bounds.

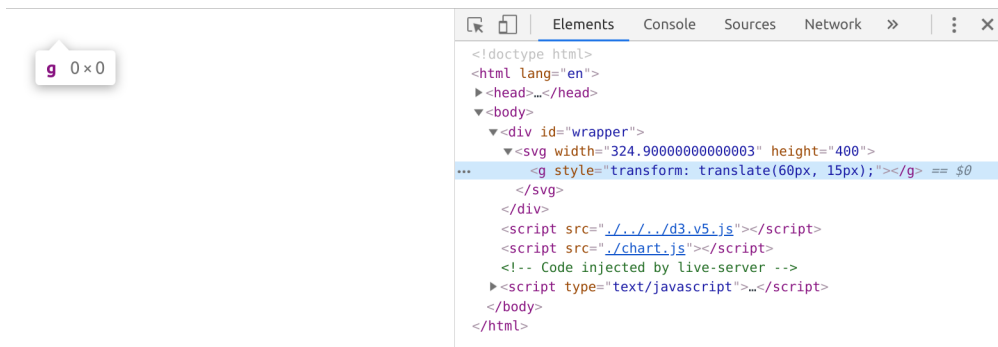
code/01-making-your-first-chart/completed/chart.js

```
36  const bounds = wrapper.append("g")
37    .style("transform", `translate(${
38      dimensions.margin.left
39    }px, ${
40      dimensions.margin.top
41    }px)`)
```

We're using backticks (``) instead of quotes (' or ") to create our `translate` string. This lets us use ES6 string interpolation — if you're unfamiliar, [read more here](https://babeljs.io/docs/en/learn/#template-strings)⁹.

⁹<https://babeljs.io/docs/en/learn/#template-strings>

If we look at our **Elements** panel, we can see our new `<g>` element.



g element

We can see that the `<g>` element size is `0px` by `0px` — instead of taking a width or height attribute, a `<g>` element will expand to fit its contents. When we start drawing our chart, we'll see this in action.

Creating our scales

Let's get back to the data.

On our y axis, we want to plot the max temperature for every day.

Before we draw our chart, we need to decide what temperatures we want to visualize. Do we need to plot temperatures over $1,000^{\circ}\text{F}$ or under 0°F ? We could hard-code a standard set of temperatures, but that range could be too large (making the data hard to see), or it could be too small or offset (cutting off the data). Instead, let's use the actual range by finding the lowest and highest temperatures in our dataset.

We've all seen over-dramatized timelines with a huge drop, only to realize that the change is relatively small. When defining an axis, we'll often want to start at 0 to show scale. We'll go over this more when we talk about types of data.

As an example, let's grab a sample day's data — say it has a maximum temperature of 55°F . We *could* draw our point 55 pixels above the bottom of the chart, but that won't scale with our `boundedHeight`.

Additionally, if our lowest temperature is below 0 we would have to plot that value below the chart! Our y axis wouldn't be able to handle all of our temperature values.

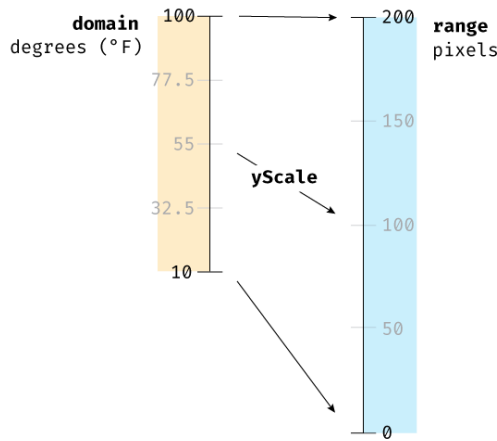
To plot the max temperature values in the correct spot, **we need to convert them into pixel space.**

d3's [d3-scale](https://github.com/d3/d3-scale)¹¹ module can create different types of scales. A scale is a **function that converts values between two domains.**

For our y axis, we want to convert values from the **temperature domain** to the **pixel domain**. If our chart needs to handle temperatures from 10°F to 100°F , a day with a max of 55°F will be halfway up the y axis.

¹¹<https://github.com/d3/d3-scale>

Let's create a scale that converts those degrees into a y value. If our y axis is 200px tall, the y scale should convert 55°F into 100, the halfway point on the y axis.



Our dataset

d3-scale¹² can handle many different types of scales - in this case, we want to use `d3.scaleLinear()` because our y axis values will be numbers that increase linearly. To create a new scale, we need to create an instance of `d3.scaleLinear()`.

code/01-making-your-first-chart/completed/chart.js

45

```
const yScale = d3.scaleLinear()
```

Our scale needs two pieces of information:

- the **domain**: the minimum and maximum input values
- the **range**: the minimum and maximum output values

Let's start with the **domain**. We'll need to create an array of the smallest and largest numbers our y axis will need to handle — in this case the lowest and highest max temperature in our dataset.

The **d3-array**¹³ module has a `d3.extent()` method for grabbing those numbers. `d3.extent()` takes two parameters:

¹²<https://github.com/d3/d3-scale>

¹³<https://github.com/d3/d3-array>

1. an array of data points
2. an accessor function which defaults to an identity function ($d \Rightarrow d$)

Let's test this out by logging `d3.extent(dataset, yAccessor)` to the console. The output should be an array of two values: the minimum and maximum temperature in our dataset. Perfect!

Let's plug that into our scale's domain:

code/01-making-your-first-chart/completed/chart.js

```
45  const yScale = d3.scaleLinear()  
46    .domain(d3.extent(dataset, yAccessor))
```

Next, we need to specify the **range**. As a reminder, the **range** is the highest and lowest number we want our scale to output — in this case, the maximum & minimum number of pixels our point will be from the x axis. We want to use our `boundedHeight` to stay within our margins. Remember, SVG y-values count from top to bottom so we want our range to start at the top.

code/01-making-your-first-chart/completed/chart.js

```
45  const yScale = d3.scaleLinear()  
46    .domain(d3.extent(dataset, yAccessor))  
47    .range([dimensions.boundedHeight, 0])
```

We just made our first scale function! Let's test it by logging some values to the console. At what y value is the freezing point on our chart?

```
console.log(yScale(32))
```

The outputted number should tell us how far away the freezing point will be from the bottom of the y axis.

If this returns a negative number, congratulations! You live in a lovely, warm place. Try replacing it with a number that “feels like freezing” to you. Or highlight another temperature that’s meaningful to you.

Let’s visualize this threshold by adding a rectangle covering all temperatures below freezing. The SVG `<rect>` element can do exactly that. We just need to give it four attributes: `x`, `y`, `width`, and `height`.



For more information about SVG elements, the [MDN docs](https://developer.mozilla.org/en-US/docs/Web/SVG/Element)¹⁴ are a wonderful resource: [here is the page for `<rect>`](https://developer.mozilla.org/en-US/docs/Web/SVG/Element/rect)¹⁵.

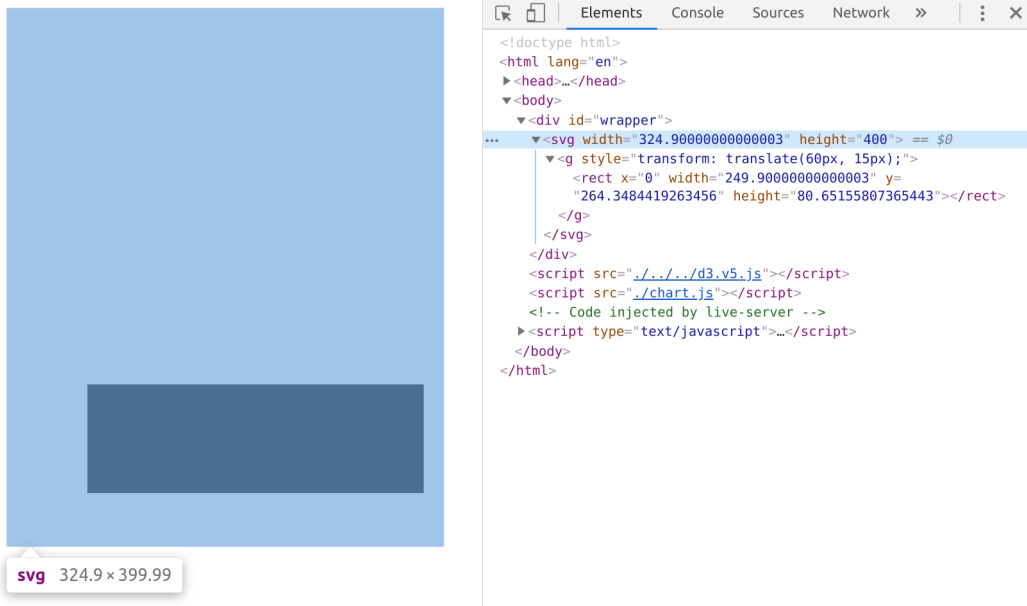
code/01-making-your-first-chart/completed/chart.js

```
49  const freezingTemperaturePlacement = yScale(32)
50  const freezingTemperatures = bounds.append("rect")
51    .attr("x", 0)
52    .attr("width", dimensions.boundedWidth)
53    .attr("y", freezingTemperaturePlacement)
54    .attr("height", dimensions.boundedHeight
55      - freezingTemperaturePlacement)
```

Now we can see a black rectangle spanning the width of our bounds.

¹⁴<https://developer.mozilla.org/en-US/docs/Web/SVG/Element>

¹⁵<https://developer.mozilla.org/en-US/docs/Web/SVG/Element/rect>



freezing point rectangle

Let's make it a frosty blue to connote "freezing" and decrease its visual importance. You can't style SVG elements with background or border — instead, we can use fill and stroke respectively. We'll discuss the differences later in more depth. As we can see, the default fill for SVG elements is black and the default stroke color is none with a width of 1px.

code/01-making-your-first-chart/completed/chart.js

```

50   const freezingTemperatures = bounds.append("rect")
51     .attr("x", 0)
52     .attr("width", dimensions.boundedWidth)
53     .attr("y", freezingTemperaturePlacement)
54     .attr("height", dimensions.boundedHeight
55       - freezingTemperaturePlacement)
56     .attr("fill", "#e0f3f3")
  
```

Let's look at the rectangle in the **Elements** panel to see how the `.attr()` methods manipulated it.

```
<rect
  x="0"
  width="1530"
  y="325.7509689922481"
  height="24.24903100775191"
  fill="rgb(224, 243, 243)"
></rect>
```

Looking good!

Some SVG styles can be set with either a CSS style or an attribute value, such as **fill**, **stroke**, and **stroke-width**. It's up to you whether you want to set them with `.style()` or `.attr()`. Once we're familiar with styling our charts, we'll apply classes using `.attr("class", "class-name")` and add styles to a separate CSS file.

In this code, we're using `.attr()` to set the fill because an attribute has a lower CSS precedence than linked stylesheets, which will let us overwrite the value. If we used `.style()`, we'd be setting an inline style which would require an `!important` CSS declaration to override.

Let's move on and create a scale for the x axis. This will look like our y axis but, since we're working with date objects, we'll use a time scale which knows how to handle date objects.

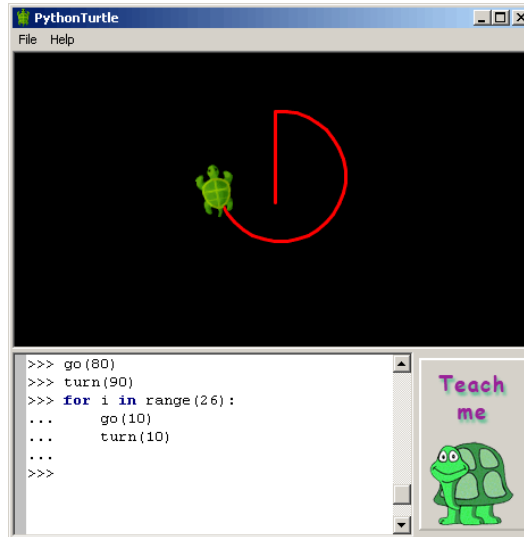
`code/01-making-your-first-chart/completed/chart.js`

```
58  const xScale = d3.scaleTime()
59    .domain(d3.extent(dataset, xAccessor))
60    .range([0, dimensions.boundedWidth])
```

Now that we have our scales defined, we can start drawing our chart!

Drawing the line

The timeline itself will be a single **path** SVG element. **path** elements take a **d** attribute (short for data) that tells them what shape to make. If you've ever played a learn-to-program game for kids, creating a **d** string is similar.



Coding turtle via <http://pythonturtle.org/>

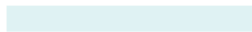
The **d** attribute will take a few commands that can be capitalized (if giving an absolute value) or lowercased (if giving a relative value):

- **M** will move to a point (followed by x and y values)
- **L** will draw a line to a point (followed by x and y values)
- **Z** will draw a line back to the first point
- ...

For example, let's draw this path:

```
bounds.append("path").attr("d", "M 0 0 L 100 0 L 100 100 L 0 50 Z")
```

We can see a new shape at the top of our chart.



d shape example

More **d** commands exist, but thankfully we don't need to learn them. d3's module **d3-shape**¹⁶ has a `d3.line()` method that will create a generator that converts data points into a **d** string.

code/01-making-your-first-chart/completed/chart.js

```
64  const lineGenerator = d3.line()
```

¹⁶<https://github.com/d3/d3-shape>

Our generator needs two pieces of information:

1. how to find an x axis value, and
2. how to find a y axis value.

We set these values with the `x` and `y` method, respectively, which each take one parameter: a function to convert a data point into an x or y value.

We want to use our accessor functions, but remember: our accessor functions return the **unscaled value**.

We'll transform our data point with both the accessor function and the scale to get the scaled value in pixel space.

code/01-making-your-first-chart/completed/chart.js

```
64  const lineGenerator = d3.line()  
65    .x(d => xScale(xAccessor(d)))  
66    .y(d => yScale(yAccessor(d)))
```

Now we're ready to add the path element to our bounds.

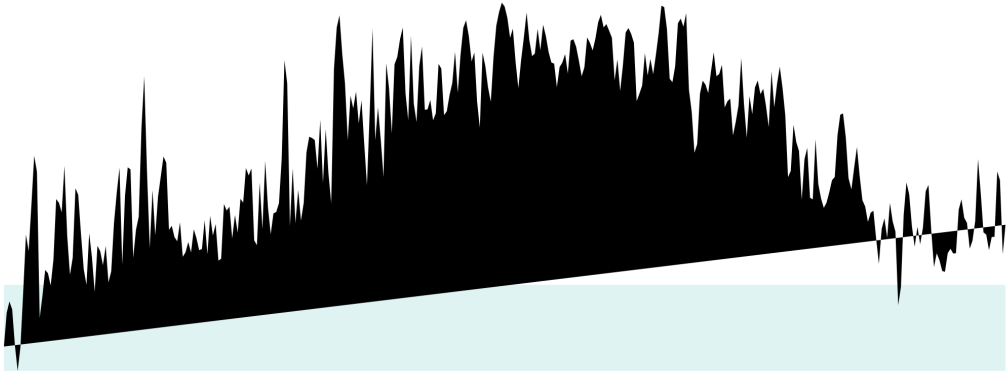
```
const line = bounds.append("path")
```

Let's feed our dataset to our line generator to create the `d` attribute and tell the line what shape to be.

```
const line = bounds.append("path")  
    .attr("d", lineGenerator(dataset))
```

Success! We have a chart with a line showing our max temperature for the whole year.

Something looks wrong, though:

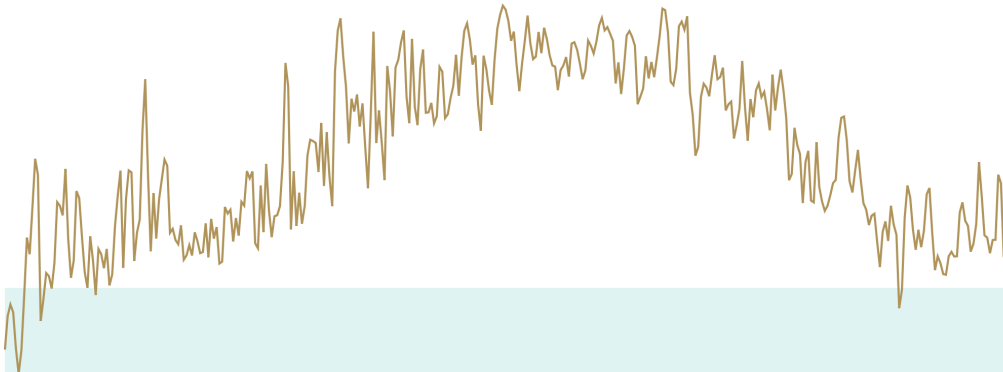


Our line!

Remember that SVG elements default to a black fill and no stroke, which is why we see this dark filled-in shape. This isn't what we want! Let's add some styles to get an orange line with no fill.

code/01-making-your-first-chart/completed/chart.js

```
68   const line = bounds.append("path")
69     .attr("d", lineGenerator(dataset))
70     .attr("fill", "none")
71     .attr("stroke", "#af9358")
72     .attr("stroke-width", 2)
```



Our line!

We're almost there, but something is missing. Let's finish up by drawing our axes.

Drawing the axes

Let's start with the y axis. d3's [d3-axis](https://github.com/d3/d3-axis)¹⁷ module has axis generator methods which will **draw an axis** *for the given scale*.

Unlike the methods we've used before, d3 axis generators will append multiple elements to the page.

¹⁷<https://github.com/d3/d3-axis>

There is one method for each orientation, which will specify the placement of labels and tick marks:

- `axisTop`
- `axisRight`
- `axisBottom`
- `axisLeft`

Following common convention, we want the labels of our y axis to be to the left of the axis line, so we'll use `d3.axisLeft()` and pass it our y scale.

`code/01-making-your-first-chart/completed/chart.js`

```
76  const yAxisGenerator = d3.axisLeft()  
77    .scale(yScale)
```

When we call our axis generator, it will create a lot of elements — let's create a `g` element to hold all of those elements and keep our DOM organized. Then we'll pass that new element to our `yAxisGenerator` function to tell it where to draw our axis.

```
const yAxis = bounds.append("g")
```

```
yAxisGenerator(yAxis)
```

This method works but it will break up our chained methods. To fix this, d3 selections have a `.call()` method that will execute the provided function with the selection as the first parameter.

We can use `.call()` to:

1. prevent saving our selection as a variable, and
2. preserve the selection for additional chaining.

Note that this code does exactly the same thing as the snippet above - we are passing the **function** `yAxisGenerator` to `.call()`, which then runs the function for us.

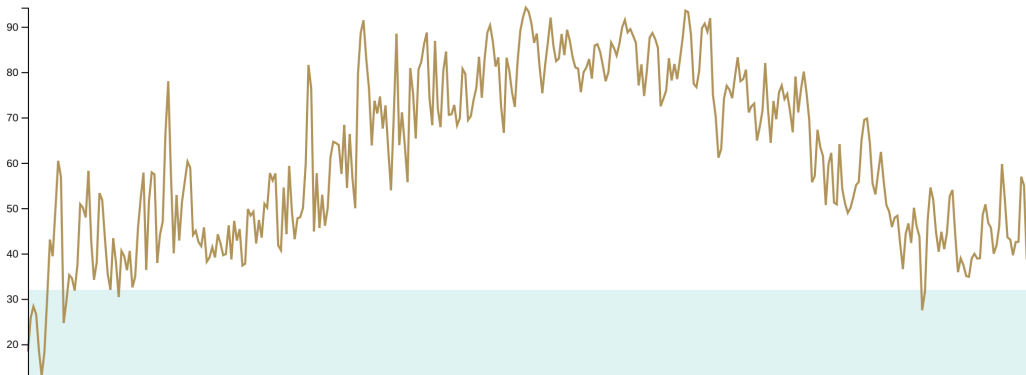
code/01-making-your-first-chart/completed/chart.js

```

79  const yAxis = bounds.append("g")
80    .call(yAxisGenerator)

```

And voila, we have our first axis!



Y axis

The small notches perpendicular to the axis are called *tick marks*. d3 has made behind-the-scenes decisions about how many tick marks to make and how far apart to draw them. We'll learn more about how to customize this later.

```

    <g fill="none" font-size="10" font-family="sans-serif" text-anchor="end">
      <path class="domain" stroke="currentColor" d="M-6,350.5H0.5V0.5H-6">
      </path>
      <g class="tick" opacity="1" transform="translate(0,337.5558785529716)">
        <line stroke="currentColor" x2="-6"></line>
        <text fill="currentColor" x="-9" dy="0.32em">30</text>
      </g>
      <g class="tick" opacity="1" transform="translate(0,309.2936046511628)">...
    </g>
    <g class="tick" opacity="1" transform="translate(0,281.03133074935397)">...
    </g>
    <g class="tick" opacity="1" transform="translate(0,252.76905684754522)">...
    </g>
    <g class="tick" opacity="1" transform="translate(0,224.5067829457364)">...
    </g>
    <g class="tick" opacity="1" transform="translate(0,196.24450904392762)">...
  </g>

```

Axis

Let's create the x axis in the same way, this time using `d3.axisBottom()`.

code/01-making-your-first-chart/completed/chart.js

```
82  const xAxisGenerator = d3.axisBottom()  
83    .scale(xScale)
```

Alright! Now let's create another `<g>` element and draw our axis.

```
const xAxis = bounds.append("g")  
    .call(xAxisGenerator)
```

We *could* `.call()` our x axis directly on our **bounds**:

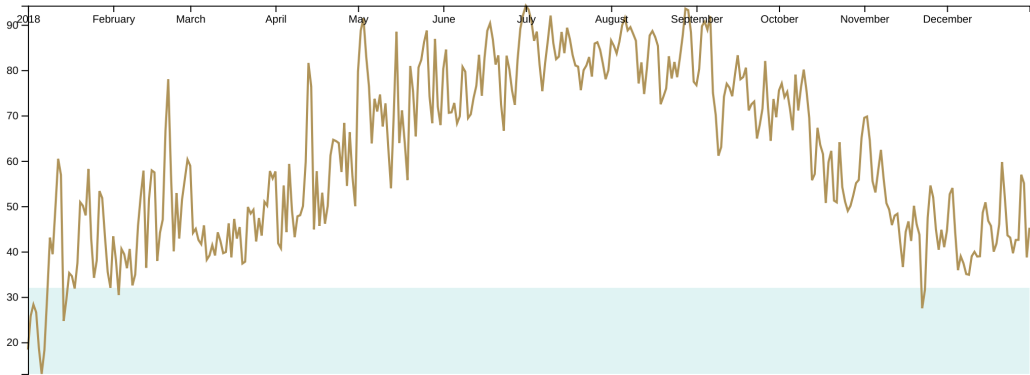
```
const xAxis = bounds.call(xAxisGenerator)
```

This would create our axis directly under our **bounds** (in the DOM).

However, it's a good idea to create a `<g>` element to contain our axis elements for three main reasons:

1. to keep our DOM organized, for debugging or exporting
2. if we want to remove or update our axis, we'll want an easy way to target all of the elements
3. modifying our whole axis at once, for example when we want to move it around.

The axis looks right, but it's in the wrong place:



x axis on top

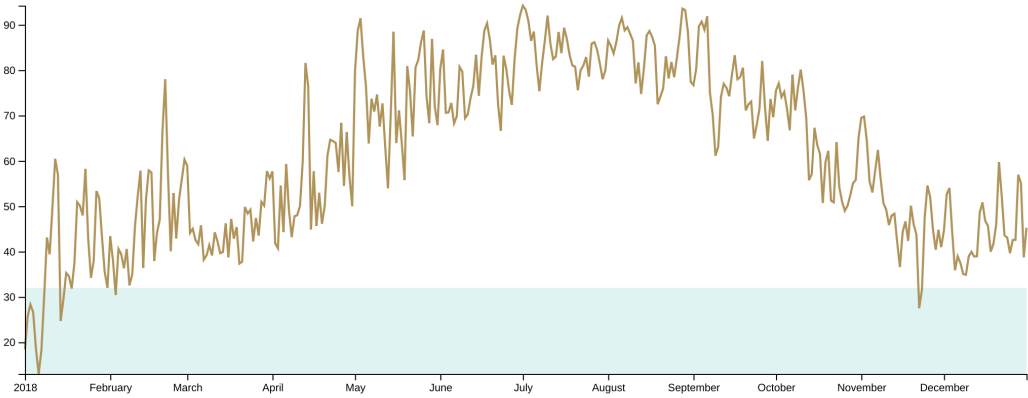
Why didn't `.axisBottom()` draw the axis in the right place? d3's axis generator functions know where to place the tick marks and tick labels relative to the axis line, but they have no idea where to place the axis itself.

To move the x axis to the bottom, we can shift the x axis group, similar to how we shifted our chart bounds using a CSS transform.

code/01-making-your-first-chart/completed/chart.js

```
85  const xAxis = bounds.append("g")
86    .call(xAxisGenerator)
87    .style("transform", `translateY(${
88      dimensions.boundedHeight
89      }px)`)
```

And just like that we're done making our first chart!



Finished line graph

Next, let's dive into making a slightly more complex chart and talk more about how d3 works for a deeper understanding of the concepts we just learned.

Making a Scatterplot

Intro

Now that we've created our first chart, let's create another chart that's a little more complex. At the end of this chapter, we'll have a deeper understanding of each step required to make a chart in d3.

There are endless questions we could ask our weather dataset — many of them ask about the relationship between different metrics. Let's investigate these two metrics:

- **dew point** is the highest temperature (°F) at which dew droplets form
- **humidity** is the amount of water vapor in the air

I would expect them to be correlated — high humidity should cause a higher dew point temperature, right? Let's dive in and find out!

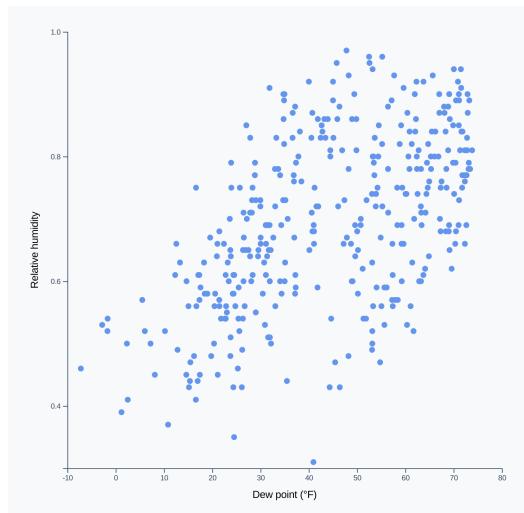
Deciding the chart type

When looking at the relationship between two metrics, a **scatterplot** is a **good choice**.

A scatterplot includes two axes:

- an **x axis** that displays one metric and
- a **y axis** that displays the other.

We'll plot **each data point** (in this case, a single day) as a dot. If we wanted to involve a third metric, we could even add another dimension by changing the color or the size of each dot.



Our Finished Scatterplot

The great thing about scatterplots is that when we're finished plotting the chart, we'll get a clear view of the relationship between the two metrics. We'll talk more about the potential patterns in **Chapter 8**.

Steps in drawing any chart

In d3, there are general steps that we need to take every time we make a chart — we briefly went through each of them in **Chapter 1** to create our line chart, but now let's create a checklist to give us a roadmap for future charts.

1. **Access data**

Look at the data structure and declare how to access the values we'll need

2. **Create chart dimensions**

Declare the physical (i.e. pixels) chart parameters

3. **Draw canvas**

Render the chart area and bounds element

4. **Create scales**

Create scales for every data-to-physical attribute in our chart

5. Draw data

Render your data elements

6. Draw peripherals

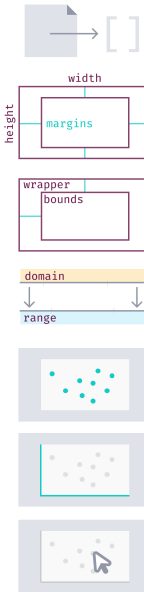
Render your axes, labels, and legends

7. Set up interactions

Initialize event listeners and create interaction behavior - we'll get to this step in Chapter 5

We have a **Chart drawing checklist** PDF in the **Advanced package** for easy reference. Feel free to print it out or save it somewhere to give you an outline in the future!

Chart drawing checklist



- ☐ **Access data**
Look at the data structure and declare how to access the values we'll need
- ☐ **Create chart dimensions**
Declare the physical (i.e. pixels) chart parameters
- ☐ **Draw canvas**
Render the **wrapper** and **bounds** element
- ☐ **Create scales**
Create scales for every data-to-physical attribute in our chart
- ☐ **Draw data**
Render your data elements
- ☐ **Draw peripherals**
Render your axes, labels, legends, annotations, etc
- ☐ **Set up interactions**
Initialize event listeners and create interaction behavior

Chart drawing checklist

Let's dig in! To start, open up the

`/code/02-making-a-scatterplot/draft/draw-scatter.js`

file and point your browser at

<http://localhost:8080/02-making-a-scatterplot/draft/>¹⁸

Remember to start your server to prevent CORS errors (`live-server`).

As usual, the completed code is in the `/code/02-making-a-scatterplot/completed/` folder, for reference.

¹⁸<http://localhost:8080/02-making-a-scatterplot/draft/>

Access data

As we saw in **Chapter 1**, this step will be quick! We can utilize `d3.json()` to grab the `my_weather_data.json` file.

`code/02-making-a-scatterplot/completed/draw-scatter.js`

```
4   let dataset = await d3.json("../..../my_weather_data.json")
```

The next part of the **Access data** step is to create our accessor functions. Let's log the first data point to the console to look at the available keys.

```
const dataset = await d3.json("../..../my_weather_data.json")  
console.table(dataset[0])
```

We can see the metrics we're interested in as `humidity` and `dewPoint`. Let's use those to define our accessor functions.

`code/02-making-a-scatterplot/completed/draw-scatter.js`

```
6   const xAccessor = d => d.dewPoint  
7   const yAccessor = d => d.humidity
```

Perfect! Now that we can access our data, we can move to the next step.

Create chart dimensions

Next up, we need to define the dimensions of our chart. Typically, scatterplots are square, with the x axis as wide as the y axis is tall. This makes it easier to look at the overall shape of the data points once they're plotted by not stretching or squashing one of the scales.

To make a square chart, we want the height to be the same as the width. We could use the same width we used in **Chapter 1** (`window.innerWidth * 0.9`), but then the chart might extend down the page, out of view on horizontal screens.

Ideally, the chart will be as large as possible while still fitting on our screen.

To fix this problem, we want to use either the height or the width of the window, **whichever one is smaller**. And because we want to leave a *little* bit of whitespace around the chart, we'll multiply the value by 0.9 (so 90% of the total width or height).

`d3-array` can help us out here with the `d3.min` method. `d3.min` takes two arguments:

1. an array of data points
2. an accessor function to grab the value from each data point

Though in this case we won't need to specify the second parameter because it defaults to an identity function and returns the value.

`code/02-making-a-scatterplot/completed/draw-scatter.js`

```
12  const width = d3.min([
13    window.innerWidth * 0.9,
14    window.innerHeight * 0.9,
15  ])
```

There is a native browser method (`Math.min`) that will also find the lowest number — why wouldn't we use that? `Math.min` is great, but there are a few benefits to `d3.min`:

- `Math.min` will count any `null`s or `undefined`s in the array as 0, whereas `d3.min` will ignore them
- `Math.min` will return `NaN` if there is a value in the array that can't be converted into a number, whereas `d3.min` will ignore it
- `d3.min` will prevent the need to create another array of values if we need to use an accessor function
- `Math.min` will return `Infinity` if the dataset is empty, whereas `d3.min` will return `undefined`
- `Math.min` uses numeric order, whereas `d3.min` uses natural order, which allows it to handle strings

You can see how `d3.min` would be preferable when creating charts, especially when using dynamic data.

Now let's use our `width` variable to define the chart dimensions:

```
let dimensions = {  
  width: width,  
  height: width,  
}
```

We were introduced to the concept of **wrapper** and **bounds** in **Chapter 1**. As a reminder:

- the **wrapper** is your entire SVG element, containing your axes, data elements, and legends
- the **bounds** live inside of the **wrapper**, containing just the data elements

Having margins around the **bounds** allows us to allocate space for our static chart elements (axes and legends) while allowing the charting area to be dynamically sized based on the available space.

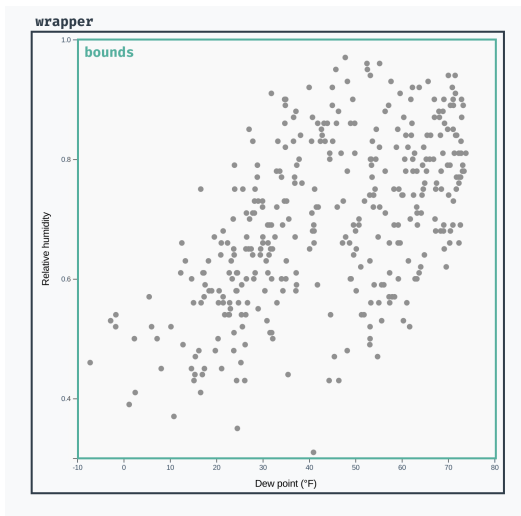


Chart terminology

We want a small top and right margin to give the chart some space. Dots near the top or right of the chart or the y axis's topmost tick label might overflow our **bounds** (because the position of the dot is technically the *center* of the dot, but the dot has a radius).

We'll want a larger bottom and left margin to create room for our axes.

code/02-making-a-scatterplot/completed/draw-scatter.js

```
16   let dimensions = {  
17     width: width,  
18     height: width,  
19     margin: {  
20       top: 10,  
21       right: 10,  
22       bottom: 50,  
23       left: 50,  
24     },  
25   }
```

Lastly, we want to define the width and height of our **bounds**, calculated from the space remaining after we add the margins.

code/02-making-a-scatterplot/completed/draw-scatter.js

```
26 dimensions.boundedWidth = dimensions.width
27   - dimensions.margin.left
28   - dimensions.margin.right
29 dimensions.boundedHeight = dimensions.height
30   - dimensions.margin.top
31   - dimensions.margin.bottom
```

You might be asking: why do we have to be explicit about the chart dimensions? Generally when developing for the web we can let elements size themselves to fit their contents or to fill the available space. That's not an option here for a few reasons:

- SVG elements scale in an unfamiliar and inconsistent way
- we need to know the width and height of the chart in order to calculate the scale outputs
- we generally want more control over the size of our chart elements — in this example, we want the width and height to be the same size, we want our dots to be large enough to see, etc

Draw canvas

Let's make some SVG elements! This step will look exactly like our line chart code. First, we find an existing DOM element (`#wrapper`), and append an `<svg>` element.

Then we use `attr` to set the size of the `<svg>` to our `dimensions.width` and `dimensions.height`. Note that these sizes are the size of the “outside” of our plot. Everything we draw next will be within this `<svg>`.

code/02-making-a-scatterplot/completed/draw-scatter.js

```
35  const wrapper = d3.select("#wrapper")
36    .append("svg")
37      .attr("width", dimensions.width)
38      .attr("height", dimensions.height)
```

Next, we create our bounds and shift them to accommodate our top & left margins.

code/02-making-a-scatterplot/completed/draw-scatter.js

```
40  const bounds = wrapper.append("g")
41    .style("transform", `translate(${
42      dimensions.margin.left
43    }px, ${
44      dimensions.margin.top
45    }px)`)
```

Above, we create a `<g>` (think “group”) element and we use the `transform` CSS property to move it to the right and down (note that the `left` margin pushes our bounds to the right, and a `top` margin pushes our bounds down).

This `bounds` is the “inner” part of our chart that we will use for our data elements.

Create scales

Before we draw our data, we have to figure out how to convert numbers from the data domain to the pixel domain.

Let's start with the x axis. We want to decide the **horizontal position** of each day's dot based on its **dew point**.

To find this position we use a **d3 scale object**, which helps us map our data to pixels. Let's create a scale that will take a dew point (temperature) and tell us *how far to the right* a dot needs to be.

This will be a *linear* scale because the input (dew point) and the output (pixels) will be numbers that increase linearly.

```
const xScale = d3.scaleLinear()
```

The concept behind scales

Remember, we need to tell our scale:

- what *inputs* it will need to handle (**domain**), and
- what *outputs* we want back (**range**).

For a simple example, let's pretend that the temperatures in our dataset range from 0 to 100 degrees.

In this case, converting from *temperature* to *pixels* is easy: a temperature of 50 degrees maps to 50 pixels because both **range** and **domain** are `[0, 100]`.

But the relationship between our data and the pixel output is rarely so simple. What if our chart was 200 pixels wide? What if we have to handle negative temperatures?

Mapping between metric values and pixels is one of the areas in which d3 scales shine.

Finding the extents

In order to create a scale, we need to pick the smallest and largest values we will handle. These numbers can be anything you want, but the standard practice is to examine your data and extract the minimum and maximum values. This way your chart will “automatically” scale according to the values in your dataset.

D3 has a helper function we can use here: `d3.extent()` that takes two parameters:

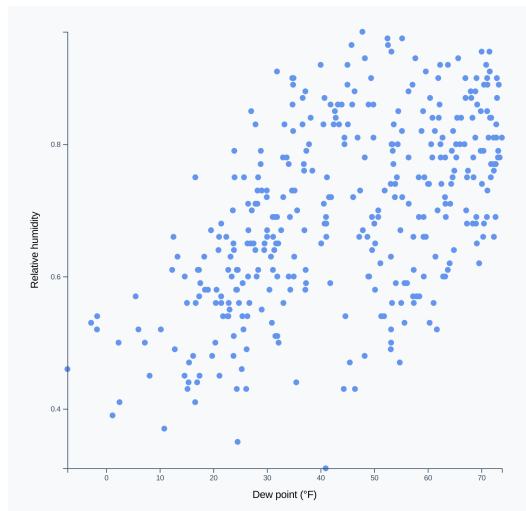
1. an array
2. an accessor function that extracts the metric value from a data point. If not specified, this defaults to an identity function `d => d`.

We’ll pass `d3.extent()` our dataset and our `xAccessor()` function and get the min and max temperatures we need to handle (in `[min, max]` format).

`code/02-making-a-scatterplot/completed/draw-scatter.js`

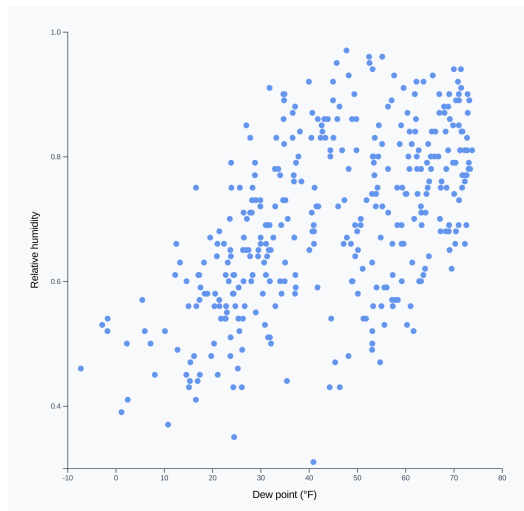
```
49  const xScale = d3.scaleLinear()  
50    .domain(d3.extent(dataset, xAccessor))  
51    .range([0, dimensions.boundedWidth])
```

This scale will create a perfectly useable chart, but we can make it slightly friendlier. With this x scale, our x axis will have a domain of `[11.8, 77.26]` — the exact min and max values from the dataset. The resulting chart will have dots that extend all the way to the left and right edges.



Finished scatterplot without nice scales

While this works, it would be easier to read the axes if the first and last tick marks were round values. Note that d3 won't even label the top and bottom tick marks of an axis with a strange domain — it might be hard to reason about a chart that scales up to 77.26 degrees. That number of decimal points gives too much unnecessary information to the reader, making them do the next step of rounding the number to a more tangible one.



Finished scatterplot with nice scales

d3 scales have a `.nice()` method that will round our scale's domain, giving our x axis friendlier bounds.

We can look at how `.nice()` modifies our x scale's domain by looking at the values before and after using `.nice()`. *Note that calling `.domain()` without parameters on an existing scale will output the scale's existing domain instead of updating it.*

```
console.log(xScale.domain())
xScale.nice()
console.log(xScale.domain())
```

With the New York City dataset, the domain changes from `[11.8, 77.26]` to `[10, 80]` — much friendlier! Let's chain that method when we create our scale.

code/02-making-a-scatterplot/completed/draw-scatter.js

```
49  const xScale = d3.scaleLinear()
50    .domain(d3.extent(dataset, xAccessor))
51    .range([0, dimensions.boundedWidth])
52    .nice()
```

Creating our y scale will be very similar to creating our x scale. The only differences are:

1. we'll be using our `yAccessor()` to grab the humidity values, and
2. we want to invert the range to make sure the axis runs bottom-to-top.

code/02-making-a-scatterplot/completed/draw-scatter.js

```
54   const yScale = d3.scaleLinear()  
55     .domain(d3.extent(dataset, yAccessor))  
56     .range([dimensions.boundedHeight, 0])  
57     .nice()
```

If we were curious about how `.nice()` modifies our y scale, we could log those values.

```
console.log(d3.extent(dataset, yAccessor))  
console.log(yScale.domain())
```

In this case, the domain changed from `[0.27, 0.97]` to `[0.2, 1]`, which will create a much friendlier chart.

Draw data

Here comes the fun part! Drawing our scatterplot dots will be different from how we drew our timeline. Remember that we had one line that covered all of the data points? For our scatter plot, we want **one element per data point**.

We'll want to use the `<circle>`¹⁹ SVG element, which thankfully doesn't need a `d` attribute string. Instead, we'll give it `cx` and `cy` attributes, which set its x and y coordinates, respectively. These position the center of the circle, and the `r` attribute sets the circle's radius (half of its width or height).

Let's draw a circle in the center of our chart to test it out.

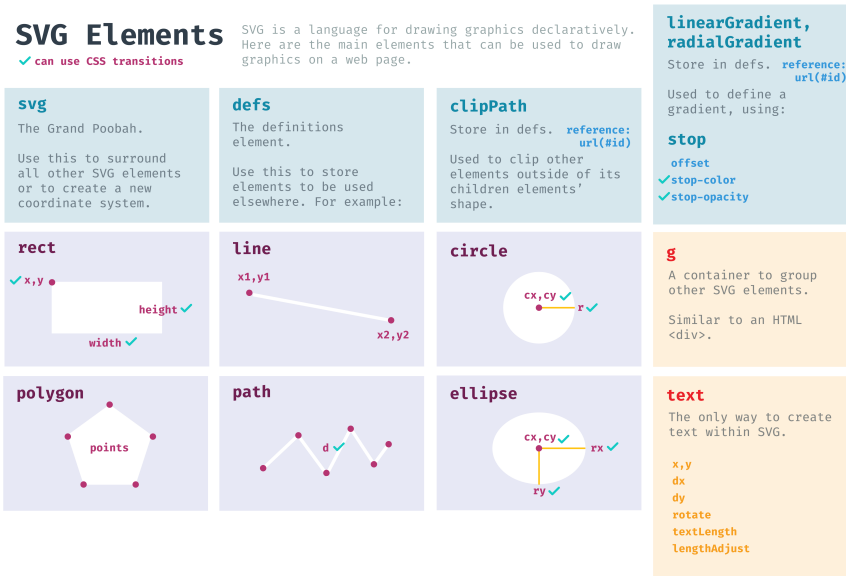
¹⁹<https://developer.mozilla.org/en-US/docs/Web/SVG/Element/circle>


```
bounds.append("circle")  
  .attr("cx", dimensions.boundedWidth / 2)  
  .attr("cy", dimensions.boundedHeight / 2)  
  .attr("r", 5)
```



Test circle

Starting to get SVG elements mixed up? No worries! Our advanced package has an **SVG elements cheat sheet** PDF to help remember what elements exist and what attributes they want. Don't worry if you don't recognize any of the elements — we'll cover them all by the end of the book.



SVG elements cheat sheet

Great! Now let's add one of those for each day.

A straightforward way of drawing the dots would be to map over each element in the dataset and append a circle to our bounds.

```
dataset.forEach(d => {
  bounds
    .append("circle")
    .attr("cx", xScale(xAccessor(d)))
    .attr("cy", yScale(yAccessor(d)))
    .attr("r", 5)
})
```

Look at that! Now we're starting to get a better sense of our data.